

PRODUCT REQUIREMENTS DOCUMENT

Unusual Mobility Behaviour Detection

for Security Surveillance

Unsupervised ML Pipeline • UCF-Crime Dataset
YOLOv8n + DeepSORT + VideoMAE + HDBSCAN

Version 1.0 • April 2026

Field	Detail
Project name	Unusual Mobility Behaviour Detection for Security Surveillance
Document version	1.0
Status	Draft — ready for implementation
Project type	Unsupervised ML research project
Dataset	UCF-Crime (1,900 surveillance videos, 13 anomaly classes)
Primary model	VideoMAE (OPear/videomae-large-finetuned-UCF-Crime)
Detection model	YOLOv8n (Ultralytics)
Tracker	DeepSORT (deep-sort-realtime==1.3.2)
Clustering	HDBSCAN (primary), DBSCAN (fallback)
Evaluation metric	Frame-level AUC (ROC)
Target AUC	>= 0.80 on UCF-Crime test split
Python version	3.10.14 (pinned)
CUDA	11.8 (torch==2.1.2+cu118)

1. Executive Summary

This document specifies the complete requirements, architecture, and implementation guidelines for an unsupervised machine learning system that detects unusual mobility behaviour in security surveillance video. The system requires no labelled training data at inference time, making it applicable to real-world surveillance environments where anomalies are rare and poorly labelled.

The pipeline fuses two complementary representations of human movement — geometric trajectory features and deep spatiotemporal video embeddings — and applies density-based clustering to separate normal from anomalous behaviour. Anomalies are ranked by a continuous score derived from cluster geometry, enabling both binary alerting and severity ranking.

The pipeline runs on the UCF-Crime benchmark, which provides 1,900 long untrimmed surveillance videos spanning 13 real-world anomaly categories. A pre-trained, domain-adapted VideoMAE model fine-tuned directly on UCF-Crime is used as the spatiotemporal feature extractor.

2. Problem Statement

2.1 Background

Modern surveillance infrastructure generates petabytes of video daily. Manual review is impractical, and rule-based systems cannot generalise across environments, lighting conditions, or crowd densities. The core challenge is detecting behaviours that deviate from a learned notion of normality — without knowing in advance what those anomalous behaviours look like.

2.2 The Problem

- Anomalous events (fights, robbery, vandalism, loitering) are rare — typically <1% of footage
- Ground-truth labels are expensive and inconsistent across cameras
- Supervised classifiers require large labelled anomaly sets per deployment environment
- Rule-based tripwires miss novel behaviour patterns
- Existing systems produce binary alerts with no severity ranking

2.3 The Goal

Build an end-to-end unsupervised pipeline that ingests raw surveillance video, tracks individual persons, extracts both motion geometry and spatiotemporal appearance features per track, and produces a continuous anomaly score per track — without any labelled training data required at deployment.

3. Project Objectives

#	Objective	Success Criterion
O1	Person detection and tracking	Stable track IDs across 30+ frames; IoU > 0.5 on benchmark clips
O2	Trajectory feature extraction	11-dimensional feature vector per track; no NaN values
O3	VideoMAE embedding extraction	1024-dim CLS token per track clip; processing < 2s/clip on GPU
O4	Feature fusion	Scaled and PCA-reduced fused vector (11 + 50 = 61-dim) per track
O5	Anomaly scoring	Continuous score in [0,1]; label-free HDBSCAN clustering
O6	Evaluation	Frame-level AUC >= 0.80 on UCF-Crime test set
O7	Visualisation	Annotated output video with bounding boxes and score overlays
O8	Reproducibility	Pinned environment; one-command setup; documented failure modes

4. Scope

4.1 In Scope

- Offline video processing (not real-time streaming)
- Person-class detection only (COCO class 0)
- Single-camera, single-scene video clips
- UCF-Crime dataset as primary benchmark
- GPU-accelerated inference (CUDA 11.8)
- Continuous anomaly score output per track
- Frame-level AUC evaluation against UCF-Crime annotations
- Annotated output video export

4.2 Out of Scope

- Real-time streaming or edge deployment
- Multi-camera scene stitching
- Non-person object detection (vehicles, bags)
- Supervised fine-tuning or labelled re-training

- Mobile or browser-based inference
- GDPR compliance or privacy masking (research context only)

5. Dataset Specification

5.1 UCF-Crime Dataset

Source	University of Central Florida — crcv.ucf.edu/projects/real-world/
Size	1,900 long untrimmed surveillance videos; 128 hours total
Anomaly classes	Abuse, Arrest, Arson, Assault, Burglary, Explosion, Fighting, Road Accident, Robbery, Shooting, Shoplifting, Stealing, Vandalism
Normal videos	800 background (normal) videos for negative bag construction
Annotations	Temporal_Anomaly_Annotation.txt — frame-level start/end per anomaly segment
Pre-processed HuggingFace	hibana2077/UCF-Crime-Dataset — 64x64 frames, every 10th frame extracted

5.2 Pre-trained Model

Model ID	OPear/videomae-large-finetuned-UCF-Crime (HuggingFace)
Architecture	VideoMAE Large — masked autoencoder with ViT-L backbone
Embedding dim	1024-dimensional CLS token (use VideoMAEModel, not ForVideoClassification)
Input format	16 frames @ 224x224 RGB, sampled per clip
Fine-tuning data	UCF-Crime subset — domain-adapted for surveillance anomaly patterns
Usage in pipeline	Feature extractor only — classification head is discarded

6. System Architecture

6.1 Pipeline Overview

The pipeline is structured as eight sequential stages, with Branch A (trajectory) and Branch B (VideoMAE) running in parallel between Stage 3 and Stage 6.

Stage	Name	Input	Output
0	Frame sampling + resize	Raw video file (.mp4/.avi)	List of BGR frames @ 5 FPS, width=640
1	YOLOv8n person detection	Frame list	Bounding boxes [x1,y1,x2,y2,conf] per frame; min 64x64 px
2	DeepSORT tracking	Bboxes + frames	Track history dict {track_id: [(cx,cy,t,bbox),...]}
3	Trajectory cleaning	Raw track history	Interpolated, length-filtered tracks (min 20 frames)
4A	Trajectory features (Branch A)	Cleaned trajectories	11-dimensional feature vector per track
4B	VideoMAE embeddings (Branch B)	Frame crops per track	1024-dimensional CLS embedding per track
5	Scaling + PCA + fusion	Feature vectors (A+B)	61-dimensional fused vector per track
6	HDBSCAN clustering	Fused vectors	Cluster labels + continuous anomaly score [0,1] per track
7	Evaluation + visualisation	Scores + annotations	AUC score, annotated video, score distribution plot

6.2 Branch A — Trajectory Features (11-dimensional)

Feature	Description	Captures
mean_speed	Mean Euclidean displacement per frame	Overall movement pace
speed_std	Standard deviation of frame-to-frame speed	Speed variability / erratic motion
max_speed	Peak instantaneous speed	Sudden bursts / running
accel_mean	Mean absolute acceleration	Smoothness of motion
accel_std	Std of acceleration	Jerkiness

Feature	Description	Captures
stop_ratio	Fraction of frames where speed < 1.0 px/frame	Loitering / standing still
total_distance	Cumulative path length in pixels	Scale of movement
displacement	Straight-line start-to-end distance	Net travel
path_efficiency	Displacement / total_distance	Directness vs. wandering
direction_variance	Variance of movement angles	Directional chaos
avg_turn_angle	Mean absolute angular change	Turning frequency

6.3 Branch B — VideoMAE Embeddings (1024-dimensional)

- Model: OPear/videomae-large-finetuned-UCF-Crime via HuggingFace transformers
- Input: 16-frame clips, cropped to person bounding box, resized to 224x224, BGR->RGB
- Extraction: CLS token from outputs.last_hidden_state[:,0,:] — shape (1024,)
- Aggregation: Mean pool over all clips in a track with stride=4
- Filtering: Crops below 64x64 px discarded; tracks with zero valid clips discarded
- Memory: torch.cuda.empty_cache() after each track to prevent OOM

6.4 Feature Fusion

- Branch A scaled independently with StandardScaler before fusion
- Branch B scaled independently with StandardScaler before PCA
- PCA applied to Branch B only: 1024 -> 50 components (fitted on normal/training set only)
- Weighted concatenation: 0.3 x Branch_A (11-dim) + 0.7 x Branch_B_PCA (50-dim)
- Final fused vector: 61 dimensions per track

7. Component Specifications

7.1 YOLOv8n Person Detector

Parameter	Value	Rationale
Package	ultralytics==8.1.47	Stable; avoids API changes in 8.2+
Model weights	yolov8n.pt (auto-downloaded)	Nano variant — fastest inference

Parameter	Value	Rationale
Detection class	0 (person only)	Scope restriction
Confidence threshold	0.4	Balance recall vs. false positives
Min bbox area	64x64 px	Prevent unusable VideoMAE crops
Output format	[x1, y1, x2, y2, conf]	Compatible with DeepSORT input

7.2 DeepSORT Tracker

Parameter	Value	Rationale
Package	deep-sort-realtime==1.3.2	Avoids TF1/TF2 conflicts in original repo
max_age	30 frames	Keep lost tracks for up to 30 frames
n_init	3 frames	Confirm track only after 3 consistent detections
nn_budget	100	Appearance descriptor memory
Output	track_id + to_ltrb()	Left-top-right-bottom bbox per confirmed track

7.3 Trajectory Cleaner

Parameter	Value	Rationale
min_len	20 frames	Discard very short / unreliable tracks
max_gap	5 frames	Interpolate small gaps due to occlusion
Interpolation	scipy.interpolate.interp1d (linear)	Smooth positional gaps

7.4 HDBSCAN Clusterer

Parameter	Value	Rationale
Package	hdbscan==0.8.33	No eps hyperparameter; robust to noise
min_cluster_size	5	Minimum tracks forming a cluster
min_samples	3	Core point threshold
metric	euclidean	Standard L2 distance in fused feature space
Anomaly label	-1	Noise / outlier tracks from HDBSCAN

Parameter	Value	Rationale
Anomaly score	Min distance to nearest cluster centroid (normalised)	Enables ranking, not just binary flag
Threshold	0.75 (configurable)	Tracks above threshold flagged as anomalous

8. Pinned Version Requirements

All versions below are tested-compatible. Do not upgrade without full regression testing.

Package	Pinned Version	Critical Reason
python	3.10.14	3.11+ causes ultralytics issues; YOLOv7 breaks on 3.12
numpy	1.26.4	CRITICAL — numpy 2.x breaks filterpy + older ultralytics + hdbscan
torch	2.1.2+cu118	Tested with transformers 4.40.2 and ultralytics 8.1.47
torchvision	0.16.2+cu118	Matched to torch 2.1.2
ultralytics	8.1.47	Stable YOLOv8 build; 8.2+ has API changes
deep-sort-realtime	1.3.2	NOT the original DeepSORT repo (TF conflicts)
filterpy	1.4.5	DeepSORT internal Kalman filter dependency
transformers	4.40.2	Full VideoMAEModel support; stable with torch 2.1
accelerate	0.29.3	Model offloading support
timm	0.9.16	Vision backbone dependency for transformers
scikit-learn	1.4.2	PCA, StandardScaler, DBSCAN
hdbscan	0.8.33	Density clustering without eps tuning
scipy	1.13.0	Trajectory interpolation
opencv-python	4.9.0.80	Frame I/O and crop extraction
Pillow	10.3.0	Image processing
matplotlib	3.8.4	Score distribution plots
tqdm	4.66.4	Progress bars
pandas	2.2.2	Results and annotation loading

Installation commands

GPU install (CUDA 11.8):

```
pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url https://download.pytorch.org/whl/cu118
```

All other dependencies:

```
pip install ultralytics==8.1.47 deep-sort-realtime==1.3.2 filterpy==1.4.5 transformers==4.40.2 accelerate==0.29.3 timm==0.9.16 scikit-learn==1.4.2 hdbscan==0.8.33 scipy==1.13.0 opencv-python==4.9.0.80 Pillow==10.3.0 matplotlib==3.8.4 tqdm==4.66.4 pandas==2.2.2
```

9. Implementation Timeline (7-Day Plan)

Day	Task	Deliverable	Risk
1	Environment setup + YOLO smoke test	Working conda env; YOLO detecting persons on 1 video	numpy version conflict
2	DeepSORT integration + track verification	Stable track IDs printed/visualised per frame	TF conflict if wrong package
3	Trajectory cleaning + Branch A features	11-dim feature vectors per track; no NaN	Very short tracks; interpolation edge cases
4	VideoMAE setup + CLS token extraction	1024-dim embedding per track on GPU	OOM; bbox too small; model download time
5	PCA + StandardScaler + feature fusion	61-dim fused vectors; scaler fitted on normal subset	Data leakage if PCA fitted on all data
6	HDBSCAN + continuous anomaly scoring	Normalised score [0,1] per track; cluster visualisation	min_cluster_size tuning; all-noise edge case
7	Evaluation + visualisation + report	AUC score; annotated video; score distribution plots	Annotation format mismatch

10. Critical Engineering Decisions

Decision	Chosen Approach	Rejected Alternative	Reason
YOLO version	YOLOv8n (ultralytics)	YOLOv7	YOLOv7 pins numpy<1.24, breaking filterpy + hdbscan
VideoMAE class	VideoMAEModel (CLS token)	VideoMAEForVideoClassification	Classification head gives logits, not embeddings

Decision	Chosen Approach	Rejected Alternative	Reason
VideoMAE weights	UCF-Crime fine-tuned (OPear/...)	Generic Kinetics weights (MCG-NJU/videomae-base)	Domain-adapted embeddings are more discriminative
Clustering	HDBSCAN (primary)	DBSCAN (fallback)	No eps tuning; handles variable density; noise-robust
Anomaly signal	Continuous score (centroid distance)	Binary HDBSCAN label only	Enables AUC evaluation and severity ranking
Branch scaling	Per-branch StandardScaler before fusion	Single scaler after fusion	Prevents high-dimensional Branch B dominating Branch A
PCA scope	Branch B only (1024->50)	Fused vector PCA	Branch A is already low-dim (11)
PCA fitting	Fit on normal training set only	Fit on all data	Avoids data leakage into anomaly cluster geometry
Bbox filter	Minimum 64x64 px	No filter	Tiny crops produce noise embeddings in VideoMAE
GPU memory	torch.cuda.empty_cache() per track	No cache clearing	Prevents OOM on long videos / many tracks
numpy version	1.26.4 (pinned)	2.x	numpy 2.x breaks filterpy Kalman filter internals

11. Evaluation Protocol

11.1 Primary Metric — Frame-level AUC

- Load UCF-Crime Temporal_Anomaly_Annotation.txt to get per-frame ground truth labels
- Map each track's anomaly score to all frames in its temporal span
- Compute `sklearn.metrics.roc_auc_score(y_true, y_score)`
- Target: AUC ≥ 0.80 (competitive with weakly-supervised baselines)

11.2 Secondary Metrics

Metric	Description	Tool
Precision @ threshold	Fraction of flagged tracks that are truly anomalous	sklearn.metrics.precision_score
Recall @ threshold	Fraction of anomalous tracks that are flagged	sklearn.metrics.recall_score
Score distribution	Histogram of anomaly scores for normal vs. anomalous tracks	matplotlib
Cluster quality	HDBSCAN cluster count; fraction of noise points (-1 labels)	Manual inspection
Track coverage	Fraction of video frames covered by confirmed tracks	Custom counter

12. Known Failure Modes and Mitigations

Failure Mode	Cause	Mitigation
numpy ImportError at runtime	numpy 2.x installed	Pin numpy==1.26.4 strictly
OOM on GPU during VideoMAE	Too many clips per track without cache clearing	Call torch.cuda.empty_cache() per track
All tracks labelled -1 by HDBSCAN	min_cluster_size too large for small dataset	Reduce min_cluster_size to 3; use fallback DBSCAN
Tiny bbox crops breaking VideoMAE	Small detections from far-away persons	64x64 px minimum filter on YOLO output
Data leakage in PCA	PCA fitted on test/anomaly data	Fit PCA only on normal training trajectories
Track ID switching (ID swap)	Fast movement or heavy occlusion in DeepSORT	Increase n_init; post-hoc track merging by proximity
VideoMAE model download failure	No internet / HuggingFace access	Pre-download with snapshot_download(); cache locally
Low AUC due to scale mismatch	Branch A and B fused without independent scaling	Always scale each branch with its own StandardScaler

13. Recommended Project Structure

```

unusual_mobility_detection/  data/          ucf_crime/          # Raw video files
annotations/                # Temporal_Anomaly_Annotation.txt  models/          yolov8n.pt
# Auto-downloaded by ultralytics  videomae_ucf/      # Cached HuggingFace model
src/          step0_sampling.py  # Frame extraction  step1_detect.py    # YOLOv8n

```

```

person detection      step2_track.py      # DeepSORT tracking      step3_clean.py
# Trajectory cleaning  step4a_traj.py      # Branch A feature extraction
step4b_vmae.py      # Branch B VideoMAE embedding      step5_fuse.py      # Scaling
+ PCA + fusion      step6_cluster.py      # HDBSCAN + anomaly scoring
step7_eval.py      # AUC + visualisation      pipeline.py      # Full end-to-end
runner      outputs/      scores/      # Per-track anomaly score CSVs
videos/      # Annotated output videos      plots/      # Score
distribution charts      requirements.txt      README.md

```

14. Acceptance Criteria

ID	Criterion	Pass Condition
AC-1	Environment setup	All packages install without conflicts in a clean conda env
AC-2	Person detection	YOLOv8n detects $\geq 90\%$ of visible persons across 10 test frames
AC-3	Track stability	Track IDs stable for ≥ 20 consecutive frames on 3 test clips
AC-4	Feature completeness	11-dim Branch A vectors contain zero NaN values for all tracks
AC-5	Embedding shape	Branch B outputs (1024,) tensor per track with no shape errors
AC-6	Fusion validity	Fused vectors are (61,) per track; scaler fitted only on normal subset
AC-7	Clustering	HDBSCAN produces ≥ 1 non-noise cluster on the training set
AC-8	Anomaly scoring	Scores normalised to [0,1]; anomalous UCF tracks score higher on average
AC-9	AUC target	Frame-level AUC ≥ 0.80 on UCF-Crime test split
AC-10	Output video	Annotated video exports with visible bounding boxes and score labels

15. References

- Sultani, W., Chen, C., Shah, M. (2018). Real-World Anomaly Detection in Surveillance Videos. CVPR 2018.
- Tong, Z. et al. (2022). VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. NeurIPS 2022.
- Wojke, N., Bewley, A., Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric. ICIAP 2017.
- Wang, C.Y. et al. (2022). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. CVPR 2023.

- Jocher, G. et al. (2023). Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- McInnes, L., Healy, J., Astels, S. (2017). HDBSCAN: Hierarchical density based clustering. JOSS.
- UCF-Crime Fine-tuned VideoMAE: <https://huggingface.co/OPear/videomae-large-finetuned-UCF-Crime>
- Singh, A.K., Singh, V. (2024). Advanced Gesture Recognition for ASD Detection: Integrating YOLOv7, Video Augmentation, and VideoMAE. arXiv:2410.09339

16. Workload Division Plan

Two-person split across the 8-stage pipeline

Stage	Description	Owner	Deliverable	Day
0	Frame sampling + resize (5 FPS, 640px)	Dev A	Frame list per video	1
1	YOLOv8n person detection (conf ≥ 0.4)	Dev A	BBoxes [x1,y1,x2,y2,conf]	2
2	DeepSORT tracking (max_age=30, n_init=3)	Dev A	Track history dict	2
3	Trajectory cleaning (min 20 frames, interp.)	Dev A	Filtered track CSVs	3
4A	Branch A: 11-dim trajectory features	Dev A	11-dim feature vectors	3–4
4B	Branch B: VideoMAE 1024-dim CLS embedding	Dev B	1024-dim embeddings per track	4
5	StandardScaler + PCA + 61-dim fusion	Dev B	61-dim fused vectors	5
6	HDBSCAN clustering + anomaly score [0,1]	Dev B	Score CSV + cluster labels	6
7	Evaluation + annotated video export	Dev B	AUC ≥ 0.80 , output video	7

Day-by-day schedule

Day	Dev A task	Dev B task	Risk
1	Conda env + YOLO smoke test on 1 UCF video	Pre-download VideoMAE model; verify CLS token shape	numpy version conflict; model download time
2	DeepSORT integration; visualise track IDs	Verify GPU memory; test embedding on 1 track	TF conflict if wrong DeepSORT package
3	Trajectory cleaner + Branch A 11-dim features	Parallel env validation; review Branch A spec	Short tracks; interpolation edge cases
4	Hand off track CSVs + feature vectors to Dev B	Receive hand-off; run Branch B on Dev A's track data	OOM; tiny bbox crops $< 64 \times 64$
5	Integration support; fix tracker edge cases	StandardScaler + PCA (fit on normal set) + fusion	Data leakage if PCA fit on all data
6	Code review + annotation format check	HDBSCAN + continuous score [0,1]; cluster visualisation	All-noise edge case; min_cluster_size tuning
7	End-to-end pipeline run (both)	AUC eval + annotated video export	Annotation format mismatch; AUC < 0.80

Shared responsibilities (both engineers)

- **Environment:** Agree on pinned versions on Day 1 — especially numpy==1.26.4 and torch==2.1.2+cu118 (critical constraints).
- **Data pipeline:** UCF-Crime download, dataset structure, and Temporal_Anomaly_Annotation.txt parsing — set up together on Day 1.
- **Integration run:** Both present for the Day 5–6 end-to-end pipeline run to catch all-noise HDBSCAN and OOM issues early.

- **Hand-off interface:** Dev A exports per-track CSV (track_id, frame indices, bbox crops path, 11-dim feature vector). Dev B consumes this as input to Stage 4B.

Note: This split assumes both developers have access to the same GPU machine (CUDA 11.8). If working on separate machines, Dev B must replicate the conda env independently before Day 4.